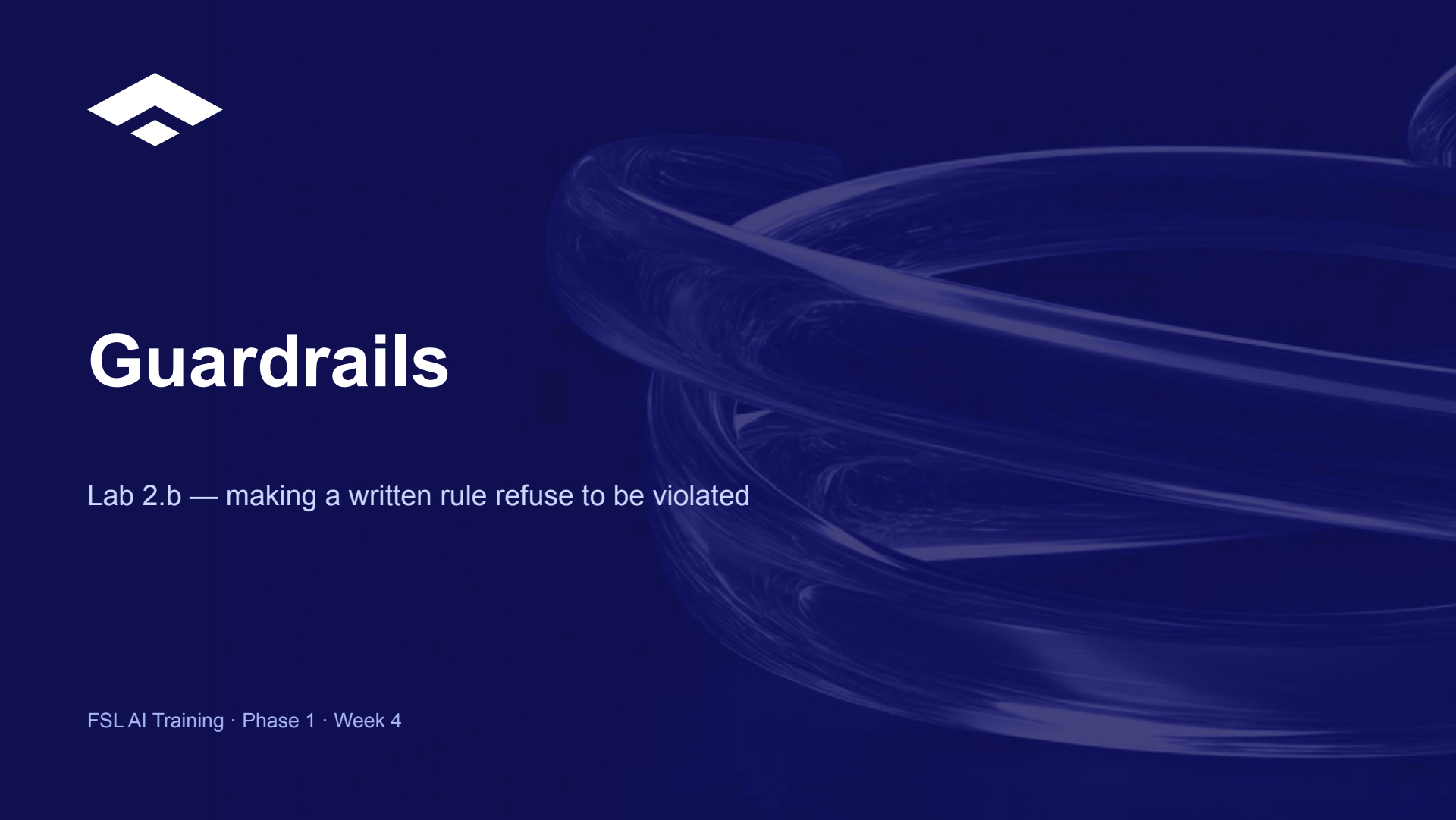


Guardrails

Lab 2.b — making a written rule refuse to be violated

FSL AI Training · Phase 1 · Week 4

TODAY

Two hours, hands-on

10 min	**Recap + why**	You have documents. Nothing enforces them
5 min	**Warm-up**	The five-minute guardrail everyone leaves with
20 min	**The five layers**	Where enforcement can live, and what each holds
25 min	**Demo — one guardrail**	Built, verified both ways, bypass documented
35 min	**Enforce your own clause**	Your rule from 2.a, at two enforcement points
15 min	**Review + hand-off**	And the artifact your team still owes

Ten minutes of slack in this block. Spend it on the verify step, not the slides.

RECAP

Where 2.a left us

One sentence: you wrote the law, and there is no police force.

- **You have an ADR.** The store boundary has a recorded reason.
- **You have an NFR you derived.** With the checks named — and the fuzzy clauses marked.
- **Both are wired into CLAUDE.md.** Claude reads them when the situation applies.
- **And nothing enforces any of it.** That is today.

The model layer cannot be the control.

Models are non-deterministic and persuadable. A rule in CLAUDE.md is a strong suggestion — it holds most of the time and fails exactly when someone asks for something reasonable-sounding that happens to violate it. If a rule matters, something deterministic has to check it. A hook is an interception point; a policy is the deterministic rule evaluated against it. Because the check does not depend on which model ran, it still holds when you swap models or add agents.

THE MAP

Five layers of enforcement

Each layer catches a different audience. You need fewer than five, but you need to know which.

Layer	Where it lives	Holds against
1 Instruction	CLAUDE.md, docs behind a trigger table	Nothing reliably — it informs
2 Client-side	.claude/settings.json permissions + hooks	The agent, while it writes
3 Commit-time	git hooks, commit lint	Anyone committing locally
4 CI gates	workflows, static analysis, validators	Anyone merging — agent or human
5 Bypass	a written escape hatch with a named approver	Nothing — it keeps 1–4 honest

This repo has 1 and 2. Its own instructor notes say so out loud rather than pretending otherwise. You will build in layer 2 and see what layer 4 costs.

WARM-UP

The five-minute guardrail

Three verdicts, not two: allow, ask, deny. No code at all.

.claude/settings.json

```
{
  "permissions": {
    "ask":  ["Bash(rm *)"],
    "deny": [
      "Edit(server/data/seed.json)",   // already in your file
      "Write(server/data/seed.json)",  // already in your file
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Bash(git clean *)",
      "Bash(npm publish *)",
      "Bash(curl * | bash)"
    ]
  }
}
```

MERGE this — do not paste over the file. The two seed rules and your hooks block must survive. Then ask Claude to force-push and watch it refuse — and confirm seed.json is still protected.

LAYER 2

Why three verdicts and not two

The middle one is the one teams forget, and it is the most useful.

ALLOW

- Pre-approved — no prompt
- Use for the safe and frequent
- Cuts prompt fatigue, which is itself a risk

ASK

- Stop and check with me
- For recoverable-but-annoying
- rm, schema changes, anything you would want to see

DENY

- Never, no prompt
- Unrecoverable or unambiguous
- Force-push, destructive infra, curl | bash

Prompt fatigue is a real failure mode: if everything asks, people stop reading and click through. Deny the truly unacceptable so that "ask" still means something.

LAYER 2

The hook already in your repo

Read this hook before you write yours — particularly its message.

DEFENSE IN DEPTH, THREE WAYS

- A Don't line in CLAUDE.md.
- A permissions.deny entry.
- A PreToolUse hook.
- All three name the same path and the same fix.

.CLAUDE/HOOKS/PROTECT-SEED.JS

```
console.error([
  'BLOCKED: Modifying seed.json
   is not allowed.',
  'Why: it is the canonical reset
   baseline every lab depends on.',
  'Instead: change data via the API
   or db.json, then reset-db.',
  'Done means: seed.json has no
   diff.',
].join('\n'));
process.exit(2);
```

SEVERITY

Block or nudge? Decide, do not default

The event you register on, plus the exit code, IS the severity decision.

Event	What exit 2 does	Use it for
PreToolUse	BLOCKS the call. stderr goes to Claude	Unrecoverable or unambiguous violations
PostToolUse	Cannot block — the tool already ran — but stderr still reaches Claude	Heuristic rules; things you should not interrupt
exit 0	Allows, silently	The 99% case
any other code	Non-blocking; the user sees a hook-error notice	Almost never on purpose

Not every guardrail should block. A gate that fires on legitimate work teaches people to route around it — and a bypassed gate is worse than none, because it looks like control.

VERIFICATION

An unverified guardrail is a belief

Two tests, always. The second one is what stops your guardrail being uninstalled.

BLOCKS THE BAD

- Try the violation deliberately
- Confirm the refusal message is useful
- Not just "it errored"

ALLOWS THE GOOD

- Run a real recent change through it
- No false positive on legitimate work
- This is the half everyone skips

STAYS TRUE

- Re-verify after every narrowing
- Fixing a false positive often reopens the hole
- Both directions, every time

Narrowing a trigger to kill a false positive is the moment a rule quietly stops catching anything. Re-verify the block case immediately after.

LAYER 5

The bypass you write down

The layer that makes the other four honest.

- **There is always one.** A label, a commit trailer, an APPROVED- comment, a person with merge rights.
- **Undocumented bypasses are the real hole.** Not the missing gate — the escape hatch nobody wrote down.
- **Name the approver.** "APPROVED-X: <reason + who> " beats a silent override every time.
- **Count how often it is used.** A bypass fired weekly is a rule that is wrong, not a team that is lax.

LAYERS 3–5

What we are teaching but not building today

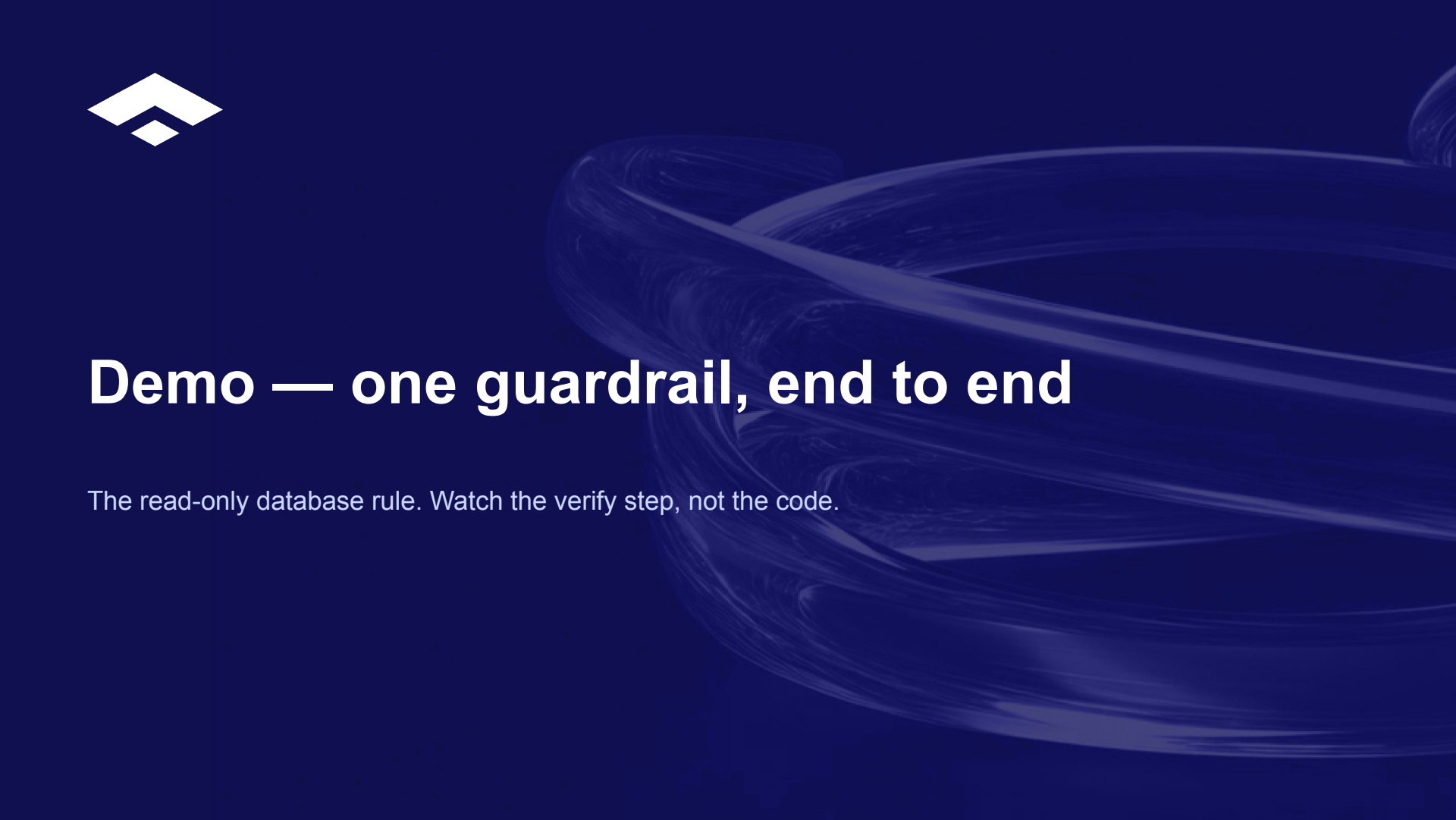
A mature corpus we have seen runs all five, including four custom static-analysis rule files and nine merge-blocking validators.

Layer	What it looks like at scale	Why not here
3 Commit-time	commit lint, pre-commit checks on staged files	Adds setup this repo does not need
4 CI gates	custom static-analysis rules, spec lint, validators that block merge	You will build one today, in your own copy
5 Bypass	APPROVED- comments with named sign-off, and telemetry on how often they fire	You will document one today

Their layer 5 is one comment convention. It is the smallest thing on the list and it is what keeps the other four trustworthy.

Rules about AI are just rules.

The same corpus governs its own AI agent with an NFR — numbered MUST clauses, a thresholds table, and eight named verification methods, covering human confirmation on destructive operations, masking sensitive data before it reaches a model, and structured output validation. There is no separate discipline for governing AI. It is the artifact you already know how to write, pointed at a new subject.



Demo — one guardrail, end to end

The read-only database rule. Watch the verify step, not the code.

DEMO · THE RULE

Read-only database unless approved

The same three-layer shape as the seed guard, on a new subject.

- **The rule.** `db.json` is generated. It gets changed through the API or a reset — not edited directly.
- **Layer 1.** A Don't line in CLAUDE.md. Already there.
- **Layer 2a.** A `permissions.deny` entry for direct writes.
- **Layer 2b.** A PreToolUse hook, so the refusal explains itself.

DEMO · THE CHECK

Twenty lines, no dependencies

Node, cross-platform, no packages. Half this room is on Windows.

.claude/hooks/protect-db.js

```
const path = filePath.replaceAll('\\', '/');

if (path.endsWith('server/data/db.json')) {
  console.error([
    'BLOCKED: db.json is generated, not edited.',
    'Why: hand-edits are lost on the next reset and',
    '  are invisible to everyone else.',
    'Instead: change data through the API, or edit',
    '  seed.json via PR and run npm run reset-db.',
    'Done means: your data change survives a reset.',
  ].join('\n'));
  process.exit(2);   // PreToolUse -> blocks
}
process.exit(0);
```

Registered on PreToolUse for Edit|Write. Note the four-part message — the same shape as the seed hook, because a refusal that does not redirect just gets worked around.

DEMO · VERIFY

Both directions, then the bypass

This is the part of the demo worth your attention.

BLOCKS

- Ask Claude to edit db.json directly
- Confirm refusal + a useful message
- Confirm it did NOT touch the file

ALLOWS

- Ask it to edit a repository file
- Confirm no refusal
- Run npm test — still green

BYPASS

- Documented in CLAUDE.md
- Instructor approval, via PR
- Not "disable the hook"

If the allow case fails, the guardrail is worse than nothing — it is now blocking work and someone will remove it by Friday.

CHECKPOINT

Checkpoint

NOBODY ADVANCES UNTIL THIS IS TRUE

Your agent is refused when it edits db.json, and succeeds when it edits a repository file.

Both directions, out loud. If you only tested the block, you have not finished — go run the allow case now.

YOUR STACK

The same pattern, somewhere you will recognise

Illustration, not today's exercise. A rule most teams have written down and almost nobody checks.

Layer	Applied to a stored-procedure convention
1 Instruction	One line: parameters only — never concatenate into executable SQL
2 Deterministic	A hook inspecting the CONTENT of the .sql being written, not just the path
4 CI gate	The same rule module, run again at merge time
5 Bypass	APPROVED-DYNAMIC-SQL: <reason + reviewer>
The honest part	The check has real false positives — a static parameterised sp_executesql is fine

Takeaway card: docs/best-practices/enforcing-a-convention.md works the whole shape through, including the clauses no gate can decide. Independent follow-up, office hours supports it.

YOUR TURN

Enforce the clause you wrote last week

Start from docs/labs/snippets/ — the plumbing is given, the policy is yours.

- **Both enforcement points.** A PostToolUse nudge when an edited file has no sibling test, and a CI gate on the PR diff.
- **Choose the severity, on purpose.** Block or nudge, per point — and write down why.
- **Verify both directions.** Refuses the violation; permits a real recent change.
- **Document the bypass.** And record a decision for every clause you could not enforce.

> fetch-depth: 0 on the checkout step, or your CI diff fails with an unhelpful error.

REVIEW

Let us compare

Fifteen minutes. Two or three screens, not everyone.

- Whose guardrail has a false positive — and how did you find out?
- Who chose nudge over block, and what was the reasoning?
- Whose bypass is documented, and who approves it?
- What went in the residue column — and whose name is against it?
- Did anyone's gate fire on a legitimate change during the exercise?

HAND-OFF

One artifact left, and it is yours

How a standard reaches every repo is a governance artifact — and we are deliberately not handing you the answer, because choosing the mechanism, the rung, and the owner IS the roll-up.

WHAT THE GUIDE GIVES YOU

- Four words that carry it: standard, drift, pin, enforce.
- CRAWL: one repo + two CLAUDE.md lines — a 4-step checklist, ~30 min.
- WALK: queryable (3 tool options, traps named) + pinned copies.
- RUN: gates, served docs, a versioned plugin — each needs an owner.
- And a fill-in template for recording what you chose.

THE DECISION TEMPLATE

```
# docs/best-practices/
#   distributing-standards.md

# what "done" looks like:
Owner:   <a person, not a team>
Rung:    crawl | walk | run
Decided: YYYY-MM-DD
Next review: YYYY-MM-DD

# ...plus: where standards live,
# how a project gets them, how you
# know you are current, enforced
# vs advisory, bypass, NOT-doing-yet
```

WRAP

What leaves this room

Ship checklist: hook registered · verified both ways · bypass documented · residue owned · peer-reviewed.

- **A guardrail that fires.** Verified both directions, with a documented bypass.
- **A residue list with names on it.** The clauses tooling cannot hold, and who accepted each.
- **The deny/ask list.** Five minutes, works in every repo you own. Do it tomorrow.
- **Three things to continue.** Port one artifact to a real repo · define how standards travel · apply the pattern to a convention of your own.